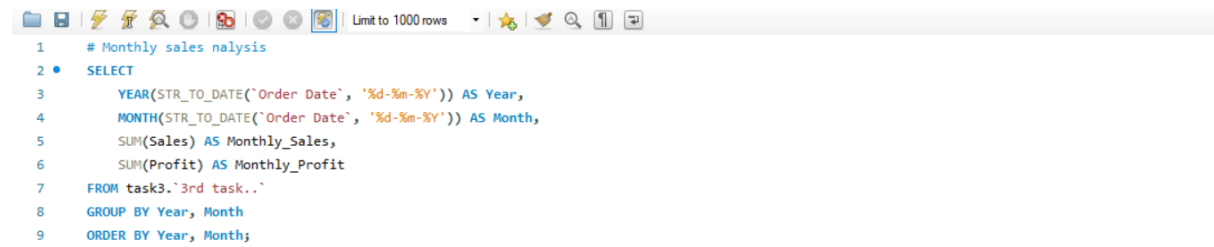


1: Monthly Performance Analysis



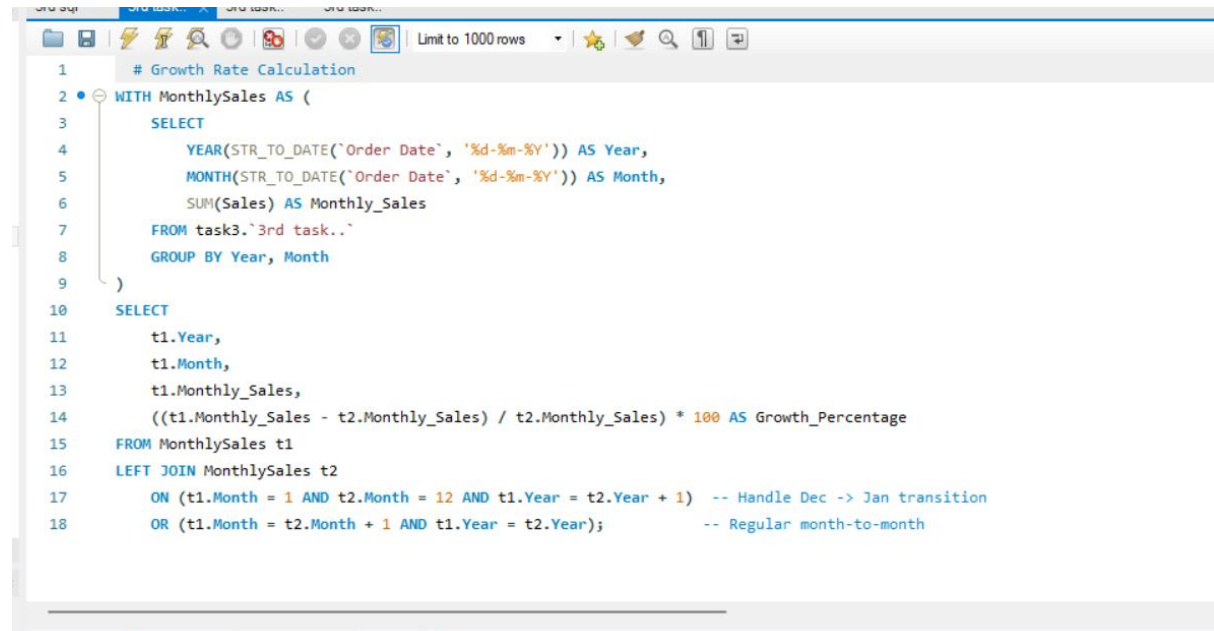
The screenshot shows a SQL query editor with a toolbar at the top. The toolbar includes icons for file operations, execution, and search. A dropdown menu shows "Limit to 1000 rows". The query text is as follows:

```
1 # Monthly sales nalysis
2 • SELECT
3     YEAR(STR_TO_DATE(`Order Date`, '%d-%m-%Y')) AS Year,
4     MONTH(STR_TO_DATE(`Order Date`, '%d-%m-%Y')) AS Month,
5     SUM(Sales) AS Monthly_Sales,
6     SUM(Profit) AS Monthly_Profit
7 FROM `task3`.`3rd task`.``
8 GROUP BY Year, Month
9 ORDER BY Year, Month;
```

#Output

# Year	Month	Monthly Sales	Monthly Profit
2014	5	1336.492	184.2804
2014	8	444.016	93.5883
2014	9	3714.304	300.7687
2014	10	211.96	8.4784
2014	11	685.18	-1.1574
2014	12	36.29	10.3751
2015	1	99.26	38.9066
2015	2	787.53	165.3813
2015	3	200.984	62.8075
2015	4	1424.425	-73.2438
2015	5	157.794	-115.7156
2015	9	4367.206	-1401.9587
2015	10	339.96	67.992
2015	11	1658.8495	-432.0946
2015	12	1228.9532	15.048
2016	1	126.138	27.7173
2016	3	103.2	36.1356
2016	4	75.88	35.6636
2016	5	566.344	146.4494
2016	6	799.77	135.92
2016	7	77.88	3.894
2016	8	2444.752	245.7878
2016	9	184.58	40.5006
2016	11	1108.65	332.9197
2016	12	317.4	103.3334
2017	4	15.552	5.4432
2017	5	301.96	33.2156
2017	6	56.994	14.1712
2017	7	71.372	-1.0196
2017	9	239.684	59.6568
2017	10	202.79	36.2566
2017	11	335.784	-28.1801
2017	12	41.96	10.9096

2: Growth Rate Calculation

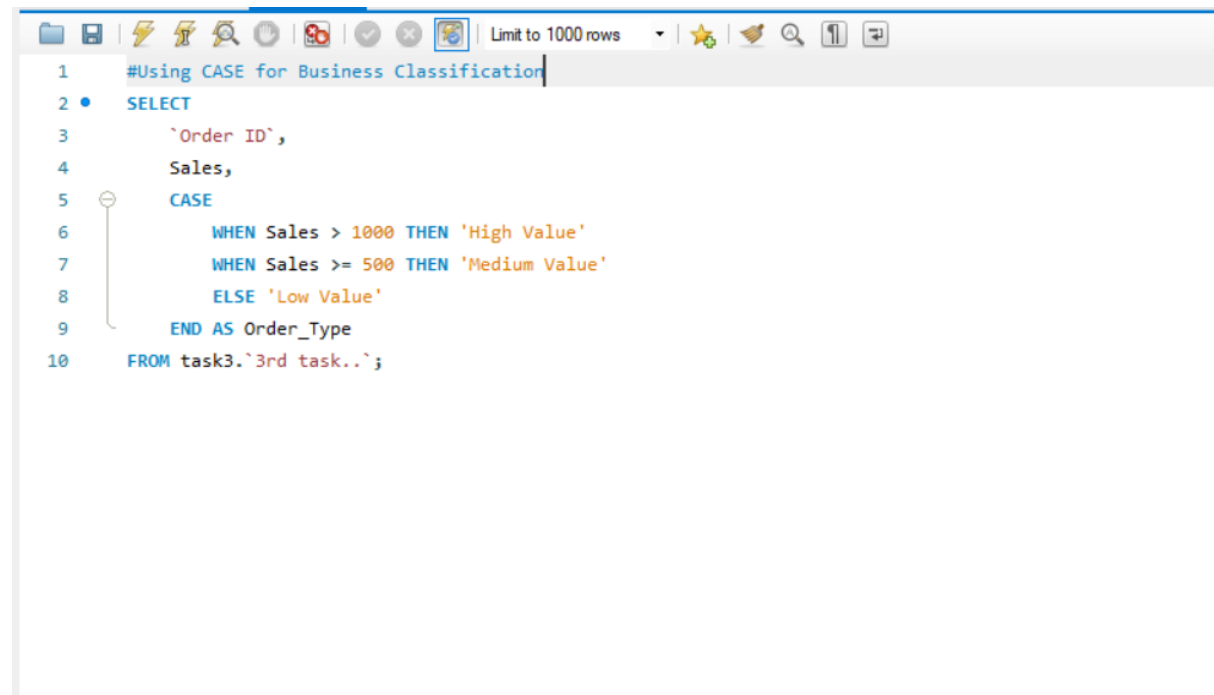


```
1 # Growth Rate Calculation
2 WITH MonthlySales AS (
3     SELECT
4         YEAR(STR_TO_DATE(`Order Date`, '%d-%m-%Y')) AS Year,
5         MONTH(STR_TO_DATE(`Order Date`, '%d-%m-%Y')) AS Month,
6         SUM(Sales) AS Monthly_Sales
7     FROM task3.`3rd task..`
8     GROUP BY Year, Month
9 )
10 SELECT
11     t1.Year,
12     t1.Month,
13     t1.Monthly_Sales,
14     ((t1.Monthly_Sales - t2.Monthly_Sales) / t2.Monthly_Sales) * 100 AS Growth_Percentage
15 FROM MonthlySales t1
16 LEFT JOIN MonthlySales t2
17     ON (t1.Month = 1 AND t2.Month = 12 AND t1.Year = t2.Year + 1) -- Handle Dec -> Jan transition
18     OR (t1.Month = t2.Month + 1 AND t1.Year = t2.Year); -- Regular month-to-month
```

#Output

# Year	Month	Monthly Sales	Growth Percentage
2016	8	2444.752	3039.126862
2016	12	317.4	-71.37058585
2015	11	1658.8495	387.9543182
2014	9	3714.304	736.5248099
2017	4	15.552	
2016	5	566.344	646.3679494
2014	11	685.18	223.2591055
2014	5	1336.492	-
2014	8	444.016	-
2016	9	184.58	-92.44994993
2017	7	71.372	25.2272169
2015	9	4367.206	
2016	1	126.138	-89.73614292
2017	10	202.79	-15.39276714
2015	12	1228.9532	-25.91532867
2016	7	77.88	-90.26220038
2017	9	239.684	-
2016	11	1108.65	-
2014	10	211.96	-94.29341271
2016	6	799.77	41.21629257
2015	4	1424.425	608.7255702
2016	4	75.88	-26.47286822
2014	12	36.29	-94.70358154
2015	3	200.984	-74.47919444
2017	11	335.784	65.5821293
2017	5	301.96	1841.615226
2015	1	99.26	173.5188757
2017	6	56.994	-81.12531461
2015	10	339.96	-92.21561795
2017	12	41.96	-87.50387154
2016	3	103.2	-
2015	2	787.53	693.4011686
2015	5	157.794	-88.92226688

3: Using CASE for Business Classification



The screenshot shows a SQL IDE window with a toolbar at the top containing icons for file operations, execution, and search. The main area displays a SQL query with line numbers 1 through 10 on the left. The query is as follows:

```
1  #Using CASE for Business Classification
2  • SELECT
3      `Order ID`,
4      Sales,
5      CASE
6          WHEN Sales > 1000 THEN 'High Value'
7          WHEN Sales >= 500 THEN 'Medium Value'
8          ELSE 'Low Value'
9      END AS Order_Type
10 FROM task3.`3rd task..`;
```

#Output

# Order ID	Sales	Order Type
US-2017-156909	71.372	Low Value
US-2017-119662	230.376	Low Value
US-2017-118038	1.248	Low Value
US-2017-118038	27.24	Low Value
US-2017-109484	5.682	Low Value
US-2015-164175	213.115	Low Value
US-2015-156867	238.896	Low Value
US-2015-156867	102.36	Low Value
US-2015-150630	3083.43	High Value
US-2015-150630	9.618	Low Value
US-2015-150630	124.2	Low Value
US-2015-150630	3.264	Low Value
US-2015-150630	86.304	Low Value
US-2015-150630	15.76	Low Value
US-2015-136476	157.794	Low Value
US-2015-134026	831.936	Medium Value
US-2015-134026	97.04	Low Value
US-2015-134026	72.784	Low Value
US-2015-118983	68.81	Low Value
US-2015-118983	2.544	Low Value
US-2015-108966	957.5775	Medium Value
US-2015-108966	22.368	Low Value
US-2014-147606	19.3	Low Value
CA-2017-161018	96.53	Low Value
CA-2017-157833	51.312	Low Value
CA-2017-155558	19.99	Low Value
CA-2017-155558	6.16	Low Value
CA-2017-146780	41.96	Low Value
CA-2017-140088	301.96	Low Value
CA-2017-139619	95.616	Low Value
CA-2017-120999	147.168	Low Value
CA-2017-119004	74.112	Low Value
CA-2017-119004	27.992	Low Value
CA-2017-119004	3.304	Low Value
CA-2017-114440	19.05	Low Value
CA-2017-114412	15.552	Low Value
CA-2017-107727	29.472	Low Value
CA-2016-169194	45	Low Value
CA-2016-169194	21.8	Low Value
CA-2016-161389	407.976	Low Value
CA-2016-159695	158.368	Low Value
CA-2016-158568	64.624	Low Value

CA-2016-158568	95.976	Low Value
CA-2016-158568	1.788	Low Value
CA-2016-152156	261.96	Low Value
CA-2016-152156	731.94	Medium Value
CA-2016-149223	77.88	Low Value
CA-2016-138688	14.62	Low Value
CA-2016-137330	19.46	Low Value
CA-2016-137330	60.34	Low Value
CA-2016-129903	23.92	Low Value
CA-2016-128867	75.96	Low Value
CA-2016-128867	27.24	Low Value
CA-2016-127208	208.16	Low Value
CA-2016-127208	16.74	Low Value
CA-2016-121755	11.648	Low Value
CA-2016-121755	90.57	Low Value
CA-2016-119823	75.88	Low Value
CA-2016-118255	45.98	Low Value
CA-2016-118255	17.46	Low Value
CA-2016-117590	1097.544	High Value
CA-2016-117590	190.92	Low Value
CA-2016-111682	208.56	Low Value
CA-2016-111682	32.4	Low Value
CA-2016-111682	319.41	Low Value
CA-2016-111682	14.56	Low Value
CA-2016-111682	30	Low Value
CA-2016-111682	48.48	Low Value
CA-2016-111682	1.68	Low Value
CA-2016-109806	20.1	Low Value
CA-2016-109806	73.584	Low Value
CA-2016-109806	6.48	Low Value
CA-2016-106075	4.616	Low Value
CA-2016-105816	15.26	Low Value
CA-2016-105816	1029.95	High Value
CA-2016-103730	47.04	Low Value
CA-2016-103730	30.84	Low Value
CA-2016-101343	77.88	Low Value
CA-2015-149734	200.984	Low Value
CA-2015-149587	12.96	Low Value
CA-2015-149587	53.34	Low Value
CA-2015-149587	32.96	Low Value
CA-2015-135545	13.98	Low Value
CA-2015-135545	25.824	Low Value
CA-2015-135545	146.73	Low Value
CA-2015-135545	79.76	Low Value
CA-2015-129476	339.96	Low Value
CA-2015-117415	113.328	Low Value

CA-2015-117415	532.3992	Medium Value
CA-2015-117415	212.058	Low Value
CA-2015-117415	371.168	Low Value
CA-2015-115742	38.22	Low Value
CA-2015-115742	75.18	Low Value
CA-2015-115742	6.16	Low Value
CA-2015-115742	89.99	Low Value
CA-2015-110457	787.53	Medium Value
CA-2015-106320	1044.63	High Value
CA-2014-167164	55.5	Low Value
CA-2014-146703	211.96	Low Value
CA-2014-143336	8.56	Low Value
CA-2014-143336	213.48	Low Value
CA-2014-143336	22.72	Low Value
CA-2014-139451	14.9	Low Value
CA-2014-139451	21.39	Low Value
CA-2014-115812	48.86	Low Value
CA-2014-115812	7.28	Low Value
CA-2014-115812	907.152	Medium Value
CA-2014-115812	18.504	Low Value
CA-2014-115812	114.9	Low Value
CA-2014-115812	1706.184	High Value
CA-2014-115812	911.424	Medium Value
CA-2014-115259	40.096	Low Value
CA-2014-115259	4.72	Low Value
CA-2014-115259	23.976	Low Value
CA-2014-115259	130.464	Low Value
CA-2014-106376	1113.024	High Value
CA-2014-106376	167.968	Low Value
CA-2014-105893	665.88	Medium Value

4: Identify Underperforming Regions

The screenshot shows a SQL IDE window with a query editor and a results grid. The query is as follows:

```
1 SELECT
2     Region,
3     SUM(Profit) AS Total_Profit
4 FROM task3.`3rd task..`
5 GROUP BY Region
6 HAVING Total_Profit < 10000;
```

The results grid displays the following data:

Region	Total_Profit
South	-134.4041
West	1194.5887
Central	173.8949
East	-1071.8486

#Output

# Region	Total Profit
South	-134.4041
West	1194.5887
Central	173.8949
East	-1071.8486